StatefulSet

Цели работы:

- изучить концепцию StatefulSet и его назначение для stateful-приложений в Kubernetes.
- понять ключевые различия между Deployment и StatefulSet.
- освоить создание и настройку StatefulSet с устойчивыми идентификаторами Pod'ов.
- научиться настраивать PersistentVolumeClaims для StatefulSet.
- освоить стратегии обновления StatefulSet.
- научиться управлять упорядоченным развертыванием и масштабированием.

`StatefulSet` — это контроллер рабочих нагрузок (workload controller) в Kubernetes, который управляет развертыванием и масштабированием набора Pod'ов и предоставляет гарантии относительно порядка и уникальности этих Pod'ов. В отличие от Deployment, StatefulSet сохраняет устойчивые идентификаторы для своих Pod'ов, что делает его идеальным для stateful-приложений, таких как базы данных, очереди сообщений и кластерные приложения.

Пример 1. Создание базового StatefulSet для приложения с постоянным хранилищем

Базовая демонстрация настройки StatefulSet с устойчивыми именами Pod'ов и индивидуальными PersistentVolumeClaims для каждого Pod'a.

Шаг 1. Создаем StatefulSet манифест

01-statefulset.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web-statefulset
spec:
  serviceName: "web-service"
  replicas: 3
  selector:
    matchLabels:
      app: nginx-stateful
  template:
    metadata:
      labels:
        app: nginx-stateful
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
        volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "hostpath"
      resources:
        requests:
          storage: 1Gi
```

Поле `volumeClaimTemplates` создает индивидуальный `PVC` для каждого `Pod'a`. `Pod'ы` будут называться: `web-statefulset-0, web-statefulset-1, web-statefulset-2.`

PVC (PersistentVolumeClaim)— это индивидуальное постоянное хранилище, которое автоматически создается для каждого Pod'a в StatefulSet и сохраняется даже при пересоздании Pod'a.

В Deployment все Pod'ы используют один общий PVC. Если Pod пересоздается, он подключается к тому же хранилищу. Нет гарантий, какой Pod получит данные

В StatefulSet каждый Pod получает свой уникальный PVC. PVC именуется по шаблону: <volumeClaimTemplateName>-<statefulsetName>-<ordinalIndex>. Данные сохраняются при перезапуске Pod'a

Шаг 2. Создаем Headless Service для управления сетевой идентичностью Pod'ов.

01-statefulset-service.yaml – блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  clusterIP: None
  selector:
    app: nginx-stateful
  ports:
  - port: 80
    targetPort: 80
```

Шаг 3. Применяем манифесты и проверяем создание StatefulSet..

```
C:\Temp>kubectl apply -f 01-statefulset.yaml
statefulset.apps/web-statefulset configured

C:\Temp>kubectl apply -f 01-statefulset-sevice.yaml
service/web-service created
```

```
C:\Temp>kubectl get statefulset
NAME              READY   AGE
web-statefulset   3/3     90s

C:\Temp>kubectl get pods -l app=nginx-stateful -w
NAME                READY   STATUS    RESTARTS   AGE
web-statefulset-0   1/1     Running   0          100s
web-statefulset-1   1/1     Running   0          97s
web-statefulset-2   1/1     Running   0          93s

C:\Temp>kubectl get pvc
NAME                    STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
www-web-statefulset-0   Bound    pvc-6710b143-1cc9-413d-99bb-142d36ee384b   1Gi        RWO            hostpath       <unset>                 114s
www-web-statefulset-1   Bound    pvc-dc542b23-c092-40d6-8b08-7dd1bcfa0306   1Gi        RWO            hostpath       <unset>                 111s
www-web-statefulset-2   Bound    pvc-ead8c8ac-34e0-412c-b623-99b2abb63c94   1Gi        RWO            hostpath       <unset>                 107s
```

Все PVC в статусе Bound (привязаны к PersistentVolume). Каждый PVC имеет объем 1Gi как указано в volumeClaimTemplates

Пример 2. Упорядоченное развертывание и масштабирование

Демонстрация последовательного создания и удаления Pod'ов в StatefulSet при масштабировании.

Шаг 1. Наблюдаем за процессом создания Pod'ов.

```
C:\Temp>kubectl get pods -l app=nginx-stateful -w
NAME                READY   STATUS    RESTARTS   AGE
web-statefulset-0   1/1     Running   0          6m11s
web-statefulset-1   1/1     Running   0          6m8s
web-statefulset-2   1/1     Running   0          6m4s
```

Шаг 2. Масштабируем StatefulSet до 5 реплик и наблюдаем процесс.

```
C:\Temp>kubectl scale statefulset web-statefulset --replicas=5
statefulset.apps/web-statefulset scaled

C:\Temp>kubectl get pods -l app=nginx-stateful -w
NAME                READY   STATUS    RESTARTS   AGE
web-statefulset-0   1/1     Running   0          6m37s
web-statefulset-1   1/1     Running   0          6m34s
web-statefulset-2   1/1     Running   0          6m30s
web-statefulset-3   1/1     Running   0          2s
web-statefulset-4   0/1     Pending   0          0s
web-statefulset-4   0/1     Pending   0          2s
web-statefulset-4   0/1     ContainerCreating   0          2s
web-statefulset-4   1/1     Running             0          4s
```

Шаг 3. Уменьшаем количество реплик до 2 и наблюдаем обратный порядок удаления

```
C:\Temp>kubectl scale statefulset web-statefulset --replicas=2
statefulset.apps/web-statefulset scaled

C:\Temp>kubectl get pods -l app=nginx-stateful -w
NAME                READY   STATUS      RESTARTS   AGE
web-statefulset-0   1/1     Running     0          7m12s
web-statefulset-1   1/1     Running     0          7m9s
web-statefulset-2   1/1     Running     0          7m5s
web-statefulset-3   0/1     Completed   0          37s
web-statefulset-3   0/1     Completed   0          37s
web-statefulset-3   0/1     Completed   0          37s
web-statefulset-2   1/1     Terminating   0          7m5s
web-statefulset-2   0/1     Completed     0          7m6s
web-statefulset-2   0/1     Completed     0          7m7s
web-statefulset-2   0/1     Completed     0          7m7s
```

Pod'ы удаляются в обратном порядке: сначала web-statefulset-4, затем web-statefulset-3

Пример 3. Стратегии обновления StatefulSet

Демонстрация различных стратегий обновления StatefulSet: RollingUpdate и OnDelete.

Так как используем те же имена, вначале лучше удалить в случае, если пересоздаем StatefulSet

```
C:\Temp>kubectl delete statefulset web-statefulset
statefulset.apps "web-statefulset" deleted from default namespace

C:\Temp>kubectl delete pvc --all
persistentvolumeclaim "www-web-statefulset-0" deleted from default namespace
persistentvolumeclaim "www-web-statefulset-1" deleted from default namespace
persistentvolumeclaim "www-web-statefulset-2" deleted from default namespace
```

Шаг 1. Настраиваем стратегию обновления в StatefulSet.

03-statefulset.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web-statefulset
spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 2
  serviceName: "web-service"
  replicas: 3
  selector:
    matchLabels:
      app: nginx-stateful
  template:
    metadata:
      labels:
        app: nginx-stateful
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
        volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
```

Добавить в Пример 1, остальная конфигурация такая же

Параметр `partition: 2` означает, что только Pod'ы с индексом >= 2 будут обновлены. Это позволяет выполнить canary-развертывание.

```
C:\Temp>kubectl apply -f 03-statefulset.yaml
statefulset.apps/web-statefulset created
```

Шаг 2. Обновляем образ контейнера и наблюдаем процесс.

```
C:\Temp>kubectl set image statefulset/web-statefulset nginx=nginx:1.26
statefulset.apps/web-statefulset image updated

C:\Temp>kubectl get pods -l app=nginx-stateful -w
NAME                 READY   STATUS              RESTARTS   AGE
web-statefulset-0    1/1     Running             0          47s
web-statefulset-1    1/1     Running             0          44s
web-statefulset-2    0/1     ContainerCreating   0          0s
web-statefulset-2    1/1     Running             0          1s
```

```
C:\Temp>kubectl describe pod web-statefulset-0 | findstr Image
    Image:          nginx:1.25
    Image ID:       docker-pullable://nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c

C:\Temp>kubectl describe pod web-statefulset-1 | findstr Image
    Image:          nginx:1.25
    Image ID:       docker-pullable://nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c

C:\Temp>kubectl describe pod web-statefulset-2 | findstr Image
    Image:          nginx:1.26
    Image ID:       docker-pullable://nginx@sha256:41b194461e4bae16f9b25d68b0976ed4735b89ca625c89aad88e1c1c3b7e8860
```

Напоминание:

Для Windows используйте `kubectl describe pod web-statefulset-2 | findstr Image`

Для Linux используйте `kubectl describe pod web-statefulset-2 | grep Image`

Pod с индексом 2 обновляется до nginx:1.26. Pod с индексами 0 и 1 остаются на nginx:1.25/ Это canary-развертывание - тестируем новую версию на одном Pod'е

Шаг 3. Меняем стратегию на OnDelete

03-statefulset.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web-statefulset
spec:
  updateStrategy:
    type: OnDelete
  serviceName: "web-service"
  replicas: 3
```

Шаг 4. Обновляем образ (но ничего не происходит!)

```
C:\Temp>kubectl apply -f 03-statefulset.yaml
statefulset.apps/web-statefulset created

C:\Temp>kubectl set image statefulset/web-statefulset nginx=nginx:1.27
statefulset.apps/web-statefulset image updated
```

Шаг 5. Проверяем Pod'ы - они НЕ обновились

```
C:\Temp>kubectl describe pod web-statefulset-0 | findstr Image
    Image:          nginx:1.25
    Image ID:       docker-pullable://nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c

C:\Temp>kubectl describe pod web-statefulset-1 | findstr Image
    Image:          nginx:1.25
    Image ID:       docker-pullable://nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c

C:\Temp>kubectl describe pod web-statefulset-2 | findstr Image
    Image:          nginx:1.25
    Image ID:       docker-pullable://nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c
```

Шаг 6. Вручную удаляем один Pod для принудительного обновления

Шаг 7. Проверяем - только удаленный Pod обновился

```
C:\Temp>kubectl delete pod web-statefulset-2
pod "web-statefulset-2" deleted from default namespace

C:\Temp>kubectl describe pod web-statefulset-2 | findstr Image
    Image:          nginx:1.27
    Image ID:
```

Сравнительная таблица стратегий:

Критерий	**RollingUpdate**	**OnDelete**
Автоматическое обновление	Да	Нет
Контроль времени обновления	Нет	Полный контроль
Использование	Dev/Test окружения	Production, критические приложения
Безопасность	Средняя	Высокая
Удобство	Высокое	Низкое

Пример 4. StatefulSet с упорядоченным запуском и готовностью

Продемонстрируем настройку проверок жизнеспособности (liveness) и готовности (readiness) для stateful-приложений, что критически важно для обеспечения надежности

Описание проблемы:в stateful-приложениях (базы данных, очереди) нельзя просто начать отправлять трафик на Pod сразу после его создания. Нужно убедиться, что приложение полностью инициализировалось, данные загружены и готовы к работе, приложение может принимать соединения

Решением выступают Probes.Kubernetes предоставляет два типа проб: ReadinessProbe - определяет, когда Pod готов принимать трафик;LivenessProbe - определяет, когда Pod "завис" и его нужно перезапустить

В примере использовались TCP probes вместо HTTP из-за возможных проблем с 404 ошибкой

Шаг 1. Создадим StatefulSet с LivenessProbe и ReadinessProbe

04-statefulset.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web-statefulset
spec:
  serviceName: "web-service"
  replicas: 3
  selector:
    matchLabels:
      app: nginx-stateful
  template:
    metadata:
      labels:
        app: nginx-stateful
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80

        # Readiness Probe - проверка готовности
        readinessProbe:
          tcpSocket:
            port: 80            # Порт для проверки
          initialDelaySeconds: 5  # Ждать 5 секунд перед первой проверкой
          periodSeconds: 10       # Проверять каждые 10 секунд
          timeoutSeconds: 2       # Таймаут проверки - 2 секунды
          successThreshold: 1     # 1 успешная проверка = Pod готов
          failureThreshold: 3     # 3 неудачи подряд = Pod не готов
```

```
        # Liveness Probe - проверка "живости"
        livenessProbe:
          tcpSocket:
            port: 80            # Порт для проверки
          initialDelaySeconds: 15 # Ждать 15 секунд перед первой проверкой
          periodSeconds: 20       # Проверять каждые 20 секунд
          timeoutSeconds: 2       # Таймаут проверки - 2 секунды
          failureThreshold: 3     # 3 неудачи подряд = перезапустить Pod

        volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html

  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "hostpath"
      resources:
        requests:
          storage: 1Gi
```

Шаг 2. Процесс запуска Pod:

```
C:\Temp>kubectl apply -f 04-statefulset.yaml
statefulset.apps/web-statefulset created

C:\Temp>kubectl get pods -l app=nginx-stateful -w
NAME                 READY   STATUS              RESTARTS   AGE
web-statefulset-0    0/1     ContainerCreating   0          2s
web-statefulset-0    0/1     Running             0          3s
web-statefulset-0    1/1     Running             0          14s
web-statefulset-1    0/1     Pending             0          0s
web-statefulset-1    0/1     Pending             0          0s
web-statefulset-1    0/1     Pending             0          2s
web-statefulset-1    0/1     ContainerCreating   0          2s
```

```
NAME                 READY   STATUS    RESTARTS   AGE
web-statefulset-0    0/1     Running   0          2s    # Создался, но не готов
web-statefulset-0    0/1     Running   0          3s    # Ждет initialDelaySeconds
web-statefulset-0    1/1     Running   0          14s    # ReadinessProbe прошла - READY
```

Шаг 3. Проверка статуса проб:

```
C:\Temp>kubectl describe pod web-statefulset-0
Name:             web-statefulset-0
Namespace:        default
Priority:         0
Service Account:  default
```

```
Containers:
  nginx:
    Container ID:   docker://7b465becddcb57b70e5471cb0d44819759c5fa22c17746e8153f3cac457766
    Image:          nginx:1.25
    Image ID:       docker-pullable://nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e6
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 28 Nov 2024 01:48:48 +0300
    Ready:          True
    Restart Count:  0
    Liveness:       tcp-socket :80 delay=15s timeout=2s period=20s #success=1 #failure=3
    Readiness:      tcp-socket :80 delay=5s timeout=2s period=10s #success=1 #failure=3
    Environment:    <none>
```

Шаг 4."Сломаем" nginx для демонстрации probes

"Ломаем" nginx в одном Pod. Наблюдаем как liveness probe обнаруживает проблему. Смотрим события.

```
C:\Temp>kubectl exec web-statefulset-1 -- nginx -s stop
2025/11/27 23:00:04 [notice] 42#42: signal process started
```

```
C:\Temp>kubectl get pods -l app=nginx-stateful -w
NAME                 READY   STATUS    RESTARTS      AGE
web-statefulset-0    1/1     Running   0             11m
web-statefulset-1    0/1     Running   1 (10s ago)   11m
web-statefulset-2    1/1     Running   0             11m
web-statefulset-1    1/1     Running   1 (13s ago)   11m
```

```
C:\Temp>kubectl get events --field-selector involvedObject.name=web-statefulset-1
LAST SEEN   TYPE      REASON            OBJECT                  MESSAGE
12m         Normal    Scheduled         pod/web-statefulset-1   Successfully assigned default/web-
110s        Normal    Pulled            pod/web-statefulset-1   Container image "nginx:1.25" alrea
110s        Normal    Created           pod/web-statefulset-1   Created container: nginx
109s        Normal    Started           pod/web-statefulset-1   Started container nginx
```

Сравнительная таблица: Deployment vs StatefulSet

Критерий	Deployment	StatefulSet
Назначение	Stateless-приложения	Stateful-приложения (БД, очереди)
Имена Pod'ов	Случайные суффиксы	Упорядоченные, устойчивые имена
Хранилище	Общие PVC	Индивидуальные PVC для каждого Pod'a
Сетевая идентичность	Неустойчивая	Устойчивые DNS имена
Порядок развертывания	Параллельный	Последовательный (0, 1, 2, ...)
Порядок удаления	Параллельный	Обратный последовательный (..., 2, 1, 0)
Обновление	RollingUpdate, Recreate	RollingUpdate, OnDelete

Задания для самостоятельного решения

Задание 12.1. Базовый StatefulSet
Создайте StatefulSet с именем `app-statefulset`, который:
- создает 3 реплики Pod'ов с именем `app-pod`
- использует образ `redis:7-alpine`
- монтирует индивидуальное хранилище 2Gi в путь `/data`
- использует Headless Service с именем `redis-service`

Проверочные команды:
```
kubectl get statefulset app-statefulset
kubectl get pods -l app=redis-app
kubectl get pvc
kubectl describe statefulset app-statefulset
```

Задание 12.2. Упорядоченное масштабирование
Для созданного StatefulSet выполните:
- масштабирование до 5 реплик и наблюдайте порядок создания Pod'ов
- масштабирование до 2 реплик и наблюдайте порядок удаления Pod'ов
- проверьте, сохранились ли PVC после удаления Pod'ов

Проверочные команды:
```
kubectl scale statefulset app-statefulset --replicas=5
kubectl get pods -l app=redis-app -w
kubectl get pvc
```

Задание 12.3. Обновление StatefulSet
Выполните обновление StatefulSet:
- Измените образ на `redis:7.2-alpine`
- Настройте стратегию обновления с partition=3
- Наблюдайте за процессом обновления

Проверочные команды:
```
kubectl rollout status statefulset app-statefulset
kubectl get pods -l app=redis-app
```

Задание 12.4. Диагностика StatefulSet
Создайте StatefulSet с преднамеренной ошибкой в конфигурации:
- укажите несуществующий StorageClass
- изучите сообщения об ошибках и события
- определите, какие Pod'ы смогли создатьсь

Проверочные команды:
```
kubectl describe statefulset <problem-statefulset>
kubectl get events --field-selector involvedObject.name=<problem-statefulset>
kubectl logs <pod-name>
```

Контрольные вопросы

1. В чем основное отличие StatefulSet от Deployment?
2. Для каких типов приложений следует использовать StatefulSet?
3. Что такое Headless Service и зачем он нужен для StatefulSet?
4. Как называются Pod'ы в StatefulSet и в каком порядке они создаются?
5. Что происходит с PersistentVolumeClaims при удалении Pod'ов StatefulSet?
6. Какие стратегии обновления доступны для StatefulSet?
7. Как параметр partition влияет на процесс обновления StatefulSet?
8. Что произойдет, если StatefulSet ссылается на несуществующий StorageClass?
9. Как можно получить доступ к конкретному Pod'у StatefulSet по DNS имени?
10. Какие команды вы бы использовали для диагностики проблем с StatefulSet?